

공동작업 #2

신상국

학습목표

팀원들이 git을 활용하여 함께 개발하는 도중 발생하는 merge conflict를 해결해 보자.

목차

1. merge conflict.....	2
1) pull.....	2
2) merge conflict.....	2
2. 실습 #1 - VSCode.....	3
1) Main.java 수정.....	3
2) commit & push.....	4
3. 실습 #2 - eclipse.....	7
1) Hello.java 수정.....	7
2) Main.java 수정.....	7
3) commit.....	8
4) push 실패.....	9
5) pull.....	10
6) commit.....	13
7) history.....	14
8) push.....	15
9) history.....	15
4. 실습 #3 - VSCode.....	17
1) Hello.java 수정.....	17
2) Main.java 수정.....	17
3) Commit.....	18
4) push 실패.....	18
5) pull.....	19
6) commit & push.....	21
7) git log --graph.....	23
5. 실습 #4 - eclipse.....	24
1) pull.....	24
2) history.....	24
6. 공동 작업.....	25
7. 요약.....	25

1. merge conflict

1) pull

다른 팀원이 remote repository에 push 한 최신 commit 들을
내 local repository에 가져와서 (fetch)
내 최신 커밋과 병합 하는 것을 (merge)
pull 이라고 한다. (fetch + merge = pull)

먼저 commit 하고 pull

수정하고 아직 커밋(commit) 안 한 파일이 내 작업 디렉토리(working tree)에 있다면,
pull 할 수 없다.
먼저 커밋한 후 pull 해야 한다.

git pull 명령 실행은 다음 3 개의 상황으로 전개된다.

- (1) 아직 push되지 않은 commit이, 내 local repository에 없다면,
최신 commit 들만 다운로드(fetch) 되고 끝난다. (실습 #3에서 git pull 명령 실행의 경우)
참고: git pull 명령 출력에 fast forward 문구가 보임
- (2) 아직 push되지 않은 commit이, 내 local repository에 있다면,
그 commit과 다운로드(fetch)된 최신 commit이
자동으로 합쳐지고 (merge)
그 결과 새 commit이 local repository에 만들어진다. (실습 #1 에서 git pull 명령 실행의 경우)
참고: git pull 명령 출력에 merge made by 문구가 보임
- (3) 아직 push되지 않은 commit이, 내 local repository에 있다면,
그 commit과 다운로드(fetch)된 최신 commit이
자동으로 합쳐져야 하는데 (merge)
이 자동 merge 작업이 실패할 수도 있다. (merge conflict)

pull이 성공했다면, merge 결과 새 커밋이 내 local repository에 등록되었으므로,
이 새 커밋을 remote repository에 push 해야 한다.

2) merge conflict

위의 pull 절차 중에서 fetch는 언제나 잘 실행되지만, merge에서 가끔 실패할 수 있다. (merge conflict)

merge는 소스 코드 줄 단위로 자동 병합하는데,
merge할 두 commit에서 수정된 소스 코드 줄이 중복되지 않는다면, merge는 성공한다.
어느 한 줄이 두 버전 모두에서 수정되었다면, merge는 실패한다. (merge conflict)

즉 어떤 팀 동료가 remote repository에 push한 commit 과
내가 local repository에만 commit하고 아직 push 하지 않은 commit 에서
어떤 소스 코드 줄을 이 두 commit 모두 수정했다면 merge conflict 가 발생한다.

merge conflict가 발생하면, 수동으로 merge 해야 한다.

2. 실습 #1 - VSCode

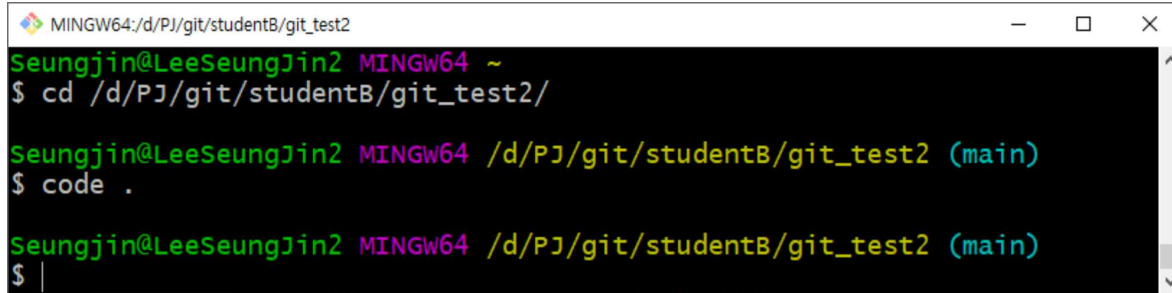
지난 강의에 이어서 git_test2 프로젝트에서 실습하자.

1) Main.java 수정

VSCode 학생의 작업

소스코드 폴더로 이동

studentB 디렉토리 아래 git_test2 디렉토리로 이동하자.



```
MINGW64:/d/PJ/git/studentB/git_test2
Seungjin@LeeSeungJin2 MINGW64 ~
$ cd /d/PJ/git/studentB/git_test2/

Seungjin@LeeSeungJin2 MINGW64 /d/PJ/git/studentB/git_test2 (main)
$ code .

Seungjin@LeeSeungJin2 MINGW64 /d/PJ/git/studentB/git_test2 (main)
$ |
```

code . 명령을 실행하여 Visual Studio Code를 열자

Main.java 수정

```
package git_test2;

public class Main {

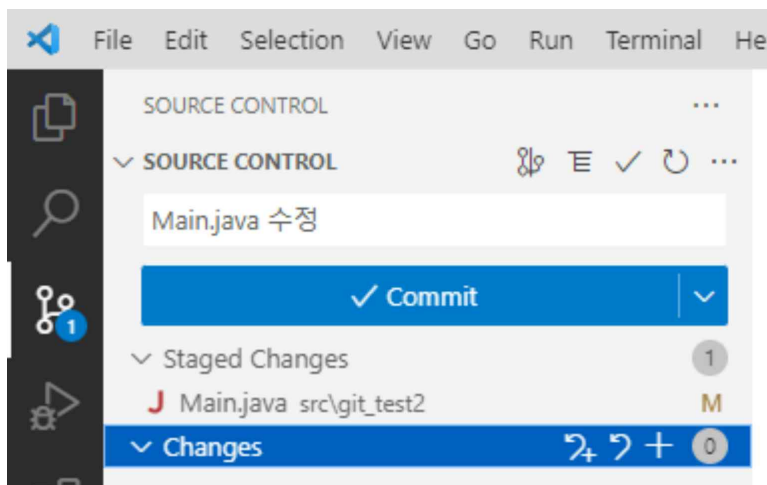
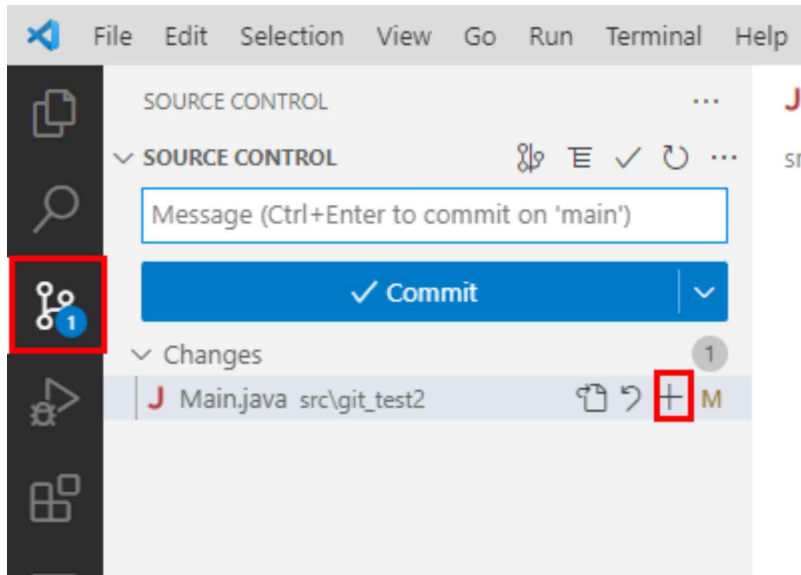
    public static void main(String[] args) {
        Hello hello = new Hello();
        hello.say();
    }

}
```

hello 변수명이 수정됨

2) commit & push

VSCode에서 커밋하기



위와 같이 메시지를 입력하고 Commit 버튼 클릭

셸 에서 커밋하기

Visual Studio Code에서 커밋하지 않고, 다음 명령으로 커밋 할 수도 있다.

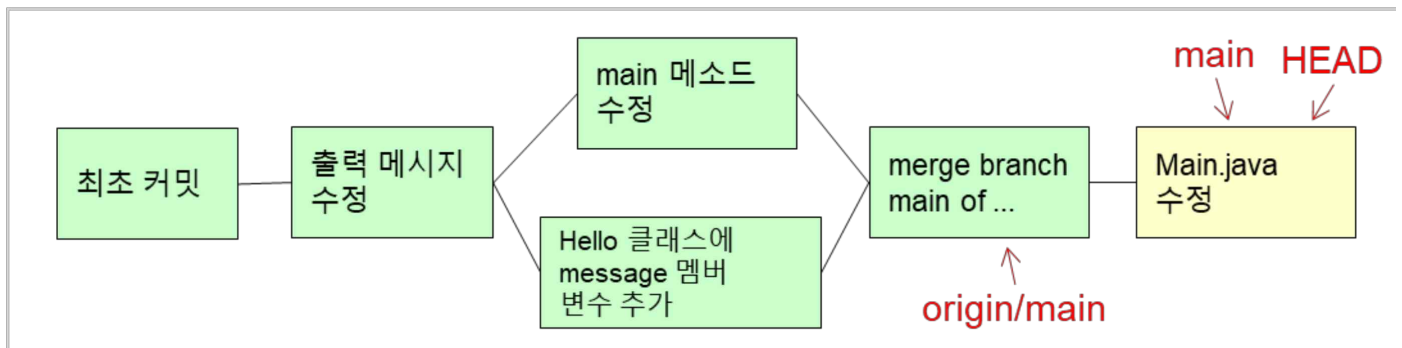
```
git commit -a -m "Main.java 수정"
```

-a 옵션 : modified 상태의 파일이 자동으로 staging area에 등록된다.

untracked 상태의 파일은 자동으로 staging area에 등록되지 않는다.

-m "...": commit message를 지정한다.

VSCode local repository

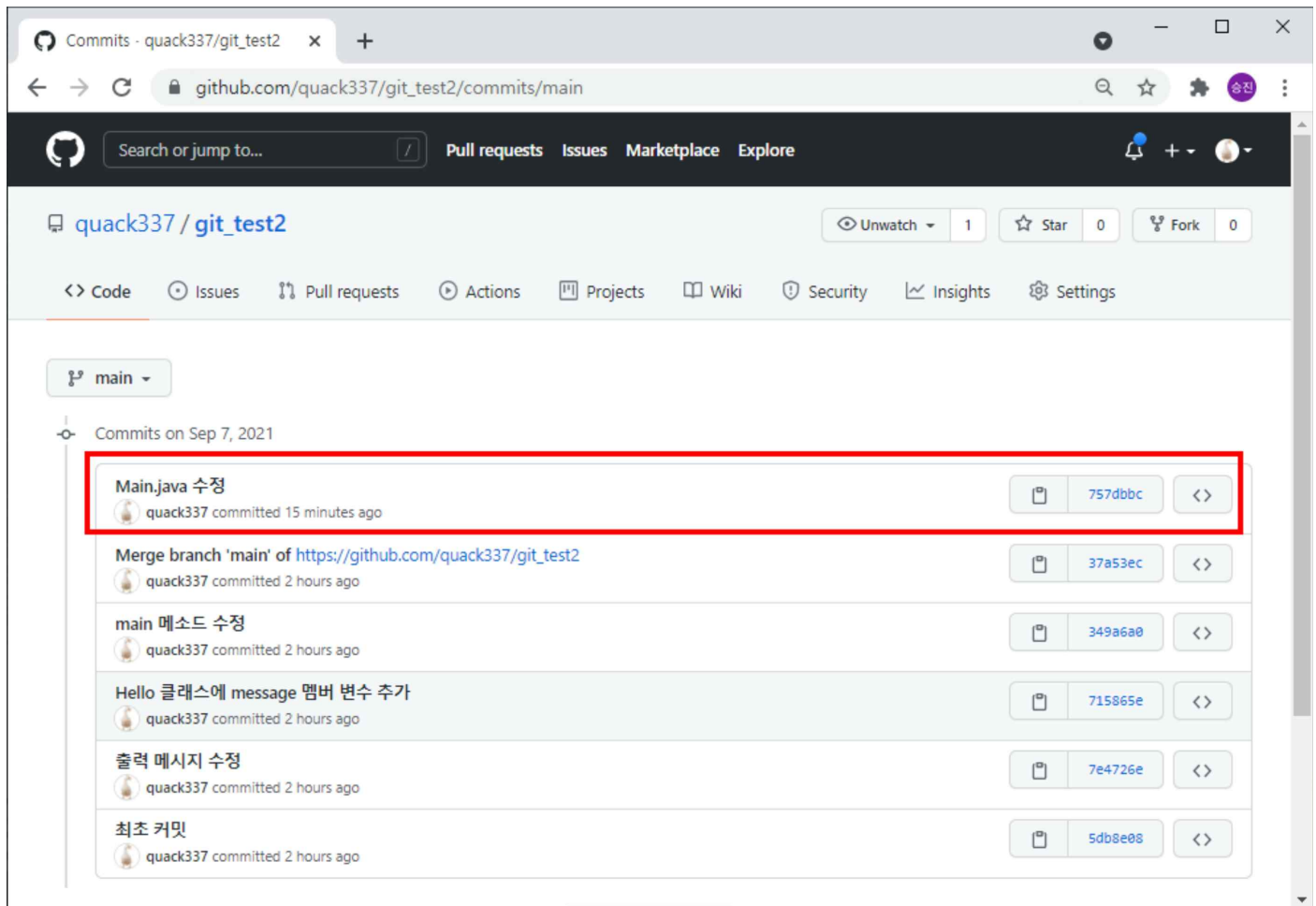


push

```
git push
```

```
Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 525 bytes | 525.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/quack337/git_test2.git
    37a53ec..757dbbc  main -> main

Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$ |
```

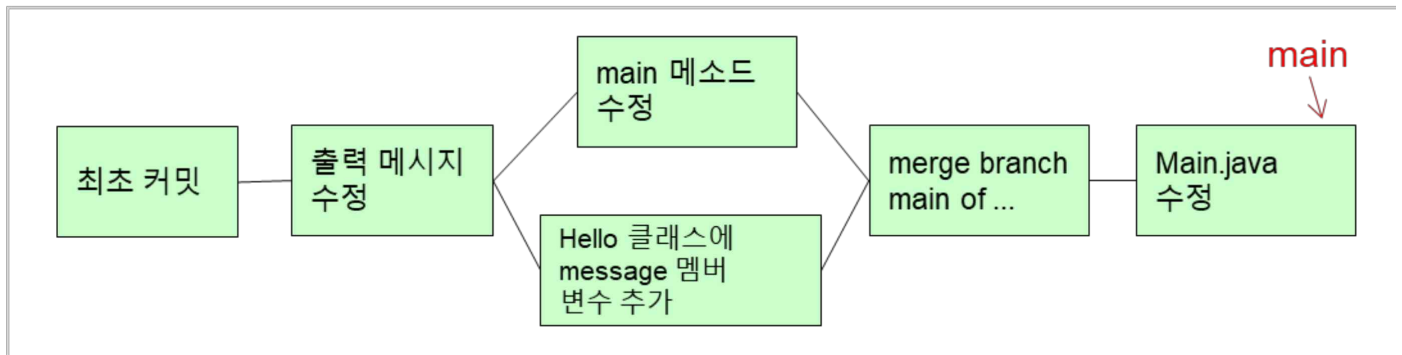


The screenshot shows the GitHub interface for the repository 'quack337/git_test2'. The 'Commits' tab is selected, displaying a list of commits on the 'main' branch. The top commit, 'Main.java 수정' by quack337, is highlighted with a red box. This commit is dated '15 minutes ago' and has the hash '757dbbc'. Below it, a merge commit is shown, followed by several other commits including 'main 메소드 수정', 'Hello 클래스에 message 멤버 변수 추가', '출력 메시지 수정', and '최초 커밋'. Each commit entry includes a commit icon, the commit message, the author's name, the time since commit, the commit hash, and a code icon.

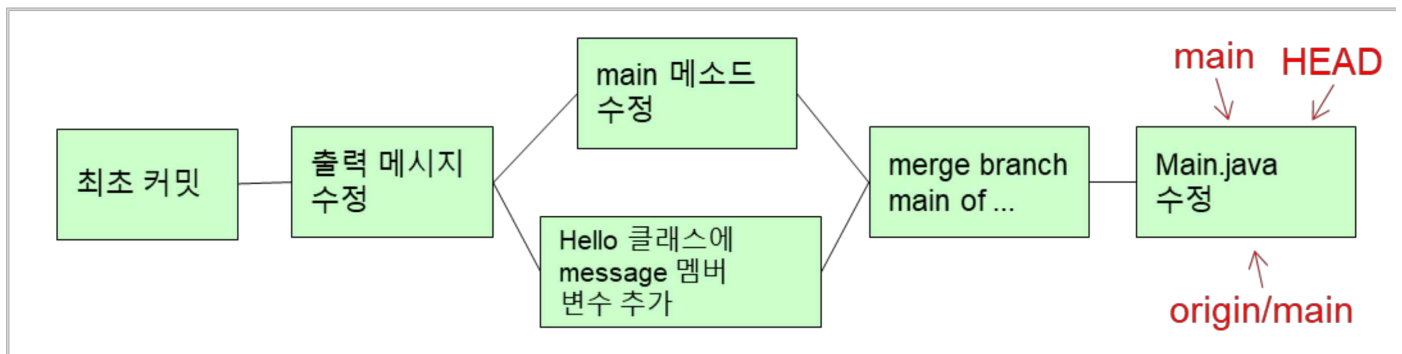
Commit Message	Author	Time	Hash	Code Icon
Main.java 수정	quack337	15 minutes ago	757dbbc	<>
Merge branch 'main' of https://github.com/quack337/git_test2	quack337	2 hours ago	37a53ec	<>
main 메소드 수정	quack337	2 hours ago	349a6a0	<>
Hello 클래스에 message 멤버 변수 추가	quack337	2 hours ago	715865e	<>
출력 메시지 수정	quack337	2 hours ago	7e4726e	<>
최초 커밋	quack337	2 hours ago	5db8e08	<>

remote repository에 새 commit 이 push 되었다.

remote repository



VSCode local repository



3. 실습 #2 – eclipse

eclipse 학생의 작업

1) Hello.java 수정

```
package git_test2;

public class Hello {

    String message;

    public Hello() {
        this("안녕하세요");
    }

    public Hello(String message) {
        this.message = message;
    }

    public void say() {
        System.out.println(message);
    }

}
```

2) Main.java 수정

```
package git_test2;

public class Main {

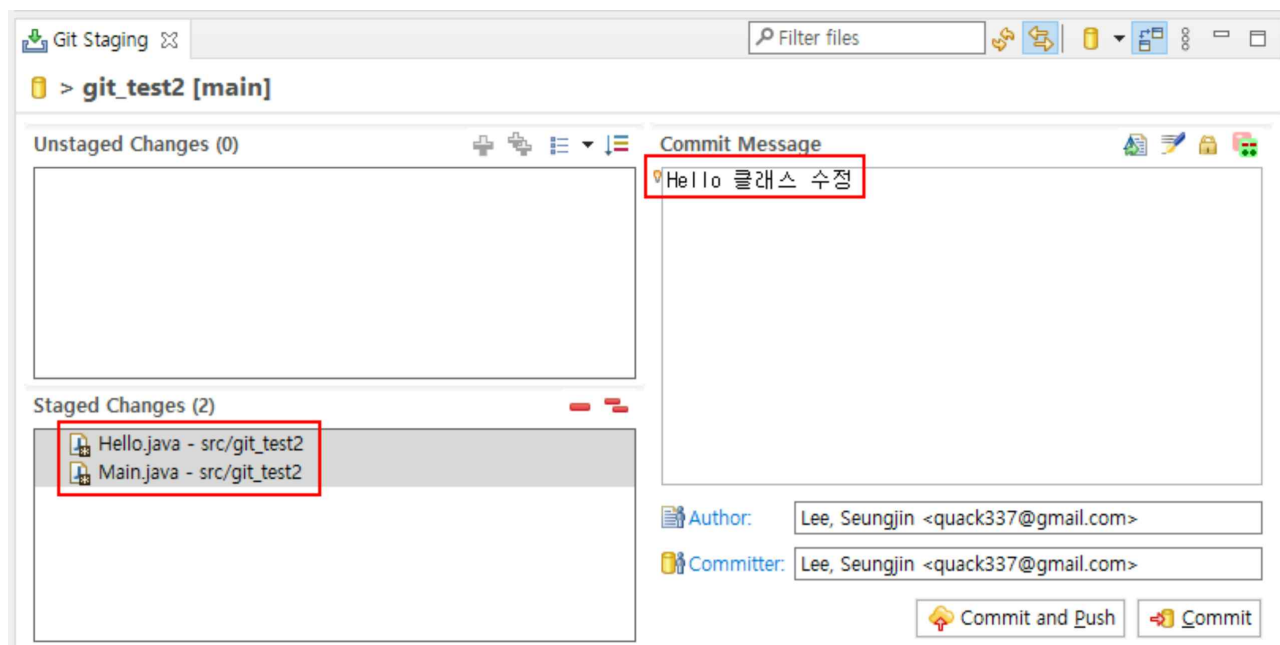
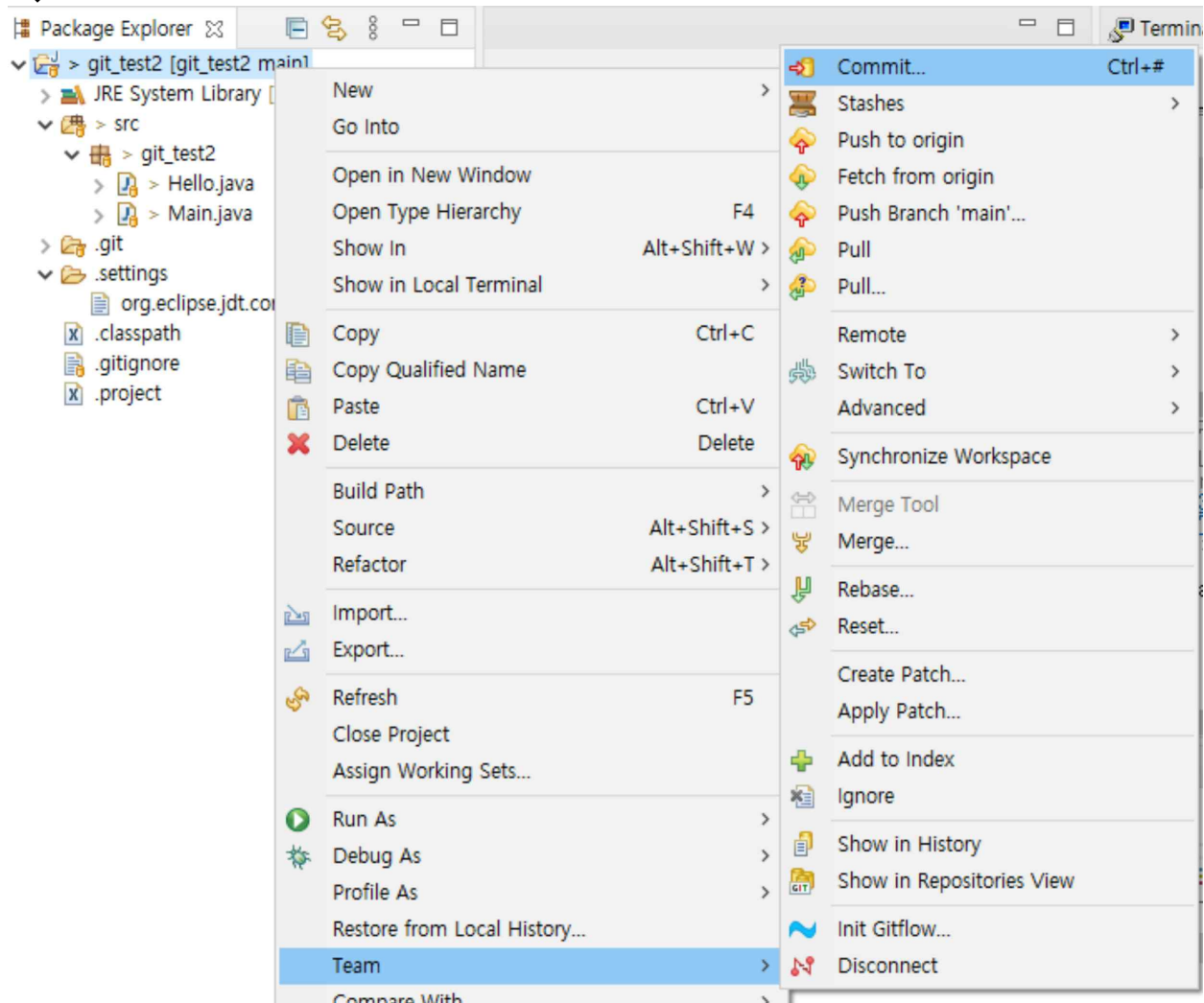
    public static void main(String[] args) {
        Hello h = new Hello("hello world");
        h.say();
    }

}
```

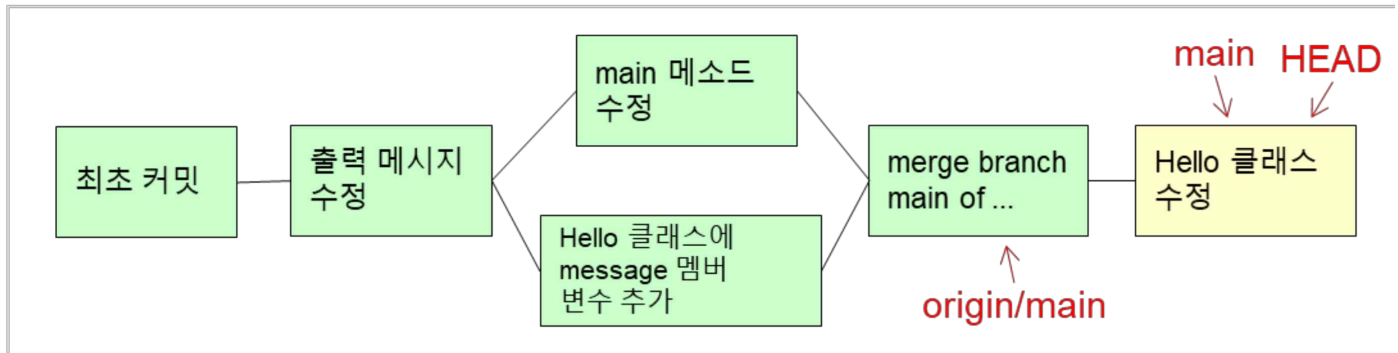
파라미터가 추가됨.

실습 #1과 동일한 줄을 수정했으므로,
git pull 할 때 merge conflict가 발생할 것이다.

3) commit



commit 클릭



4) push 실패

git push

```

Terminal
MINGW64:/c/PJ/STS/java/git_test2

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git push
To https://github.com/quack337/git_test2.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/quack337/git_test2.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$
  
```

git push 명령이 실패했다. (rejected)

VSCode학생이 push한 커밋을 먼저 다운로드 해야 한다.

git push 하기 전에 먼저 git pull 명령을 실행한 후 git push 해야 하기 때문이다.

git pull 먼저

push 하기 전에 먼저 pull을 해야 한다.

5) pull

git pull

```
Terminal
MINGW64:/c/PJ/STS/java/git_test2

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git push
To https://github.com/quack337/git_test2.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/quack337/git_test2.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git pull
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 5 (delta 0), reused 5 (delta 0), pack-reused 0
Unpacking objects: 100% (5/5), 505 bytes | 26.00 KiB/s, done.
From https://github.com/quack337/git_test2
  37a53ec..757dbbc  main    -> origin/main
Auto-merging src/git_test2/Main.java
CONFLICT (content): Merge conflict in src/git_test2/Main.java
Automatic merge failed; fix conflicts and then commit the result.

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main|MERGING)
$
```

git pull 명령도 실패한다.

src/git_test2/Main.java 파일에서 merge conflict 발생했다.

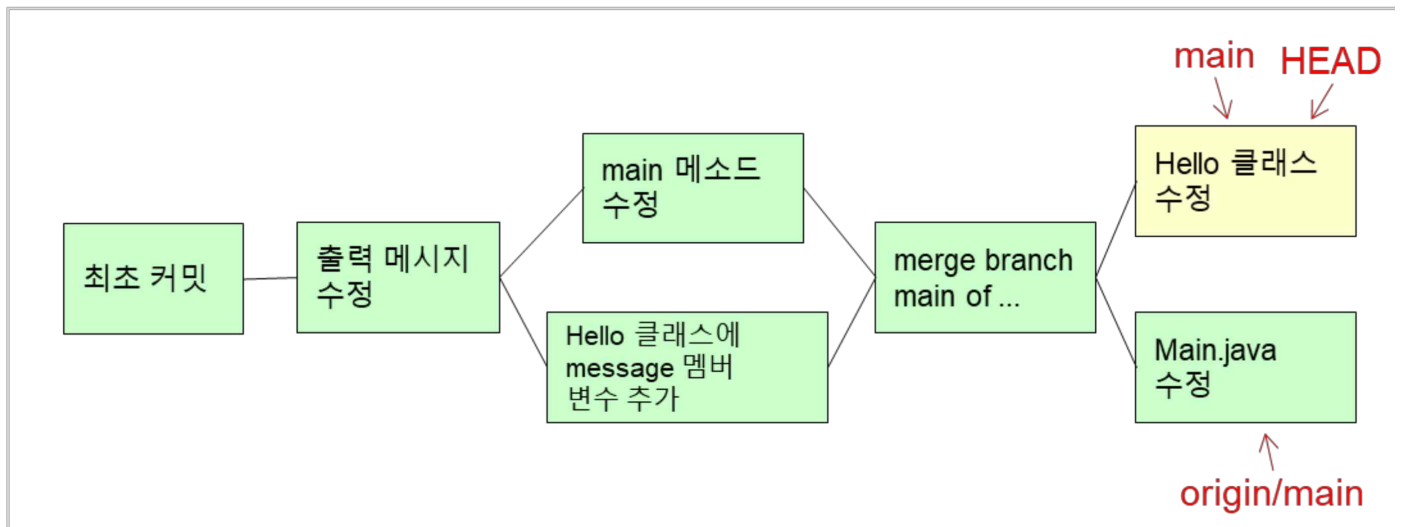
자동 merge 실패했으므로, 수동 merge 하고 commit 해야한다.

```
Package Explorer
git_test2 [git_test2|Conflicts main ↑ 1 ↓ 1]
├── JRE System Library [JavaSE-16]
├── src
│   ├── git_test2
│   │   ├── Hello.java
│   │   └── Main.java
│   ├── .settings
│   ├── .classpath
│   ├── .gitignore
│   └── .project
```

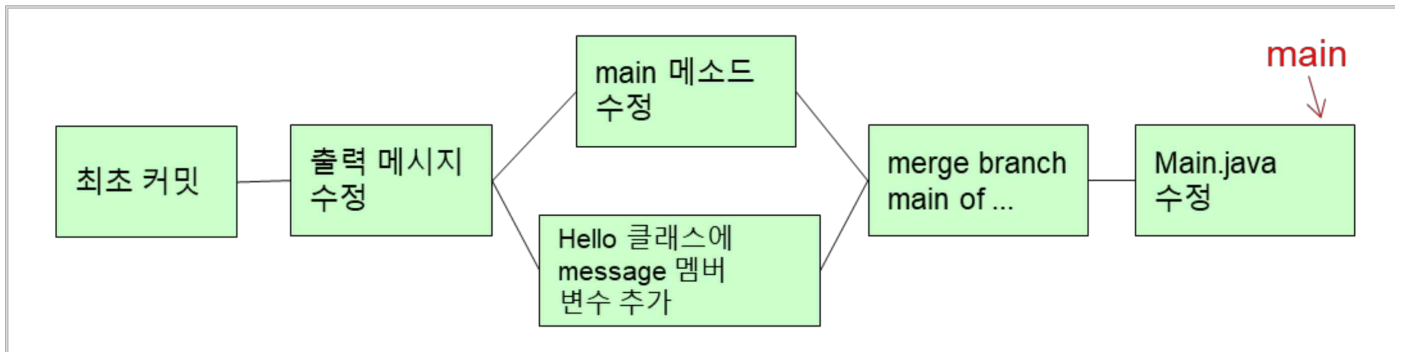
merge conflict 발생한 파일에 빨간색 아이콘이 보인다.

> Main.java

eclipse GUI local repository



remote repository



Main.java

```
package git_test2;

public class Main {

    public static void main(String[] args) {
<<<<<<< HEAD
        Hello h = new Hello("hello world");
        h.say();
=====
        Hello hello = new Hello();
        hello.say();
>>>>>>> 757dbbc31da27755a61648b874b85468314d2d6a
    }

}
```

merge conflict 발생한 소스 코드이다.

```
<<<<<<< HEAD
    내가 수정하여 local repository에만 commit 한 소스코드
=====
    remote repository 에서 다운로드 한 commit 의 소스코드
>>>>>>> 757dbbc31da27755a61648b874b85468314d2d6a
```

이 두 commit의 자동 merge 가 실패했다.
이 두 commit의 소스코드를 합쳐서 수정해야 한다.
빨간색으로 칠한 부분은 지워야 한다.

Main.java 수정

merge conflict 발생한 부분을 다음과 같이 수정하자.

```
package git_test2;

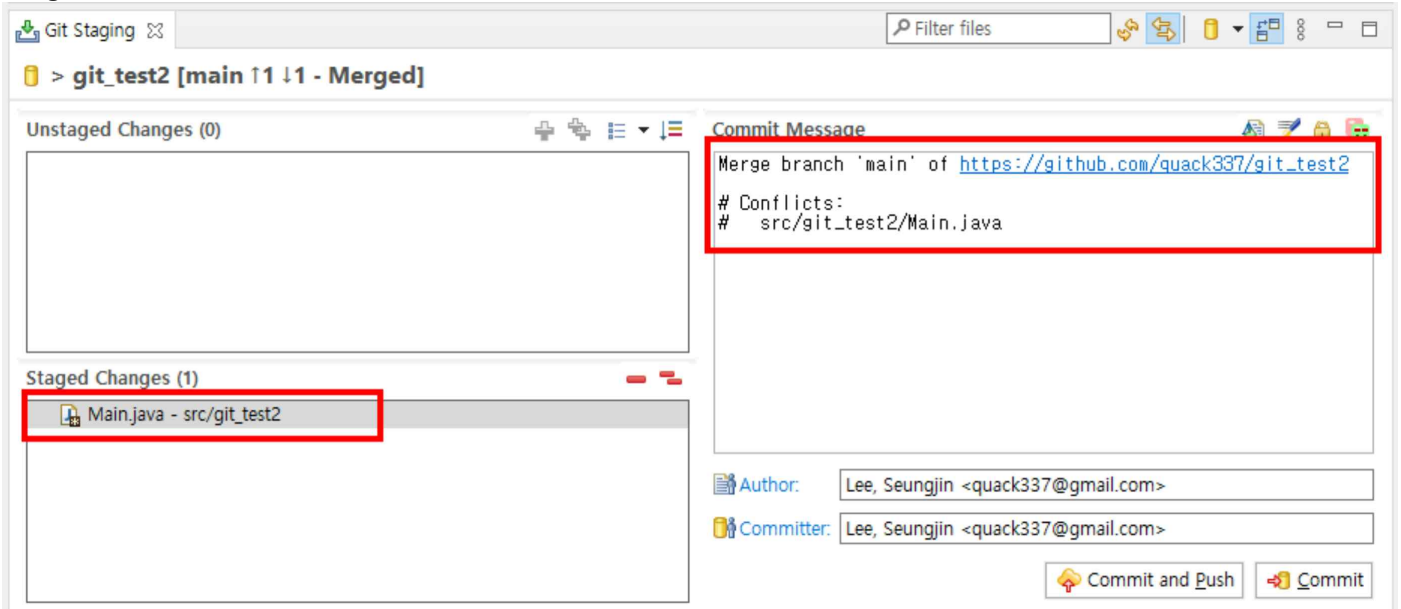
public class Main {

    public static void main(String[] args) {
        Hello hello = new Hello("hello world");
        hello.say();
    }

}
```

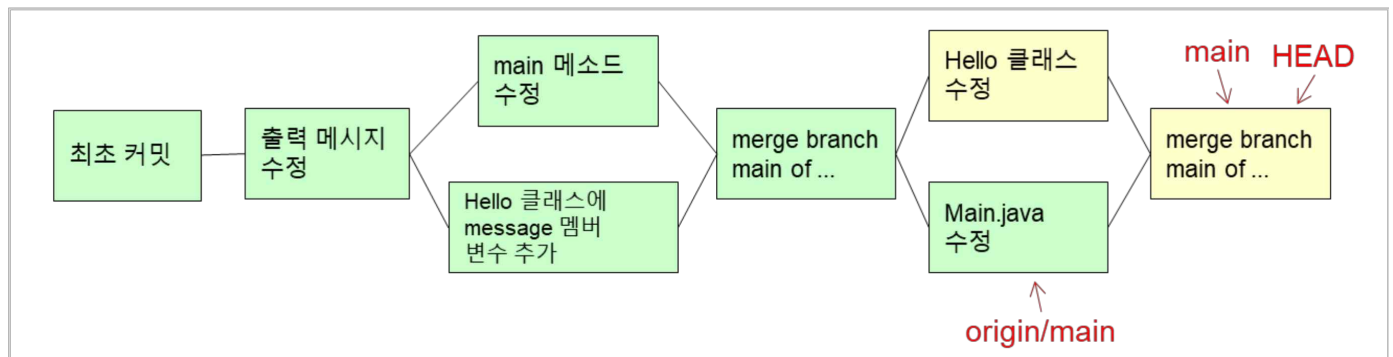
6) commit

merge conflict를 해결한 새 버전을 commit 하자.



Main.java를 Staged Changes 영역으로 옮기고
Commit 클릭

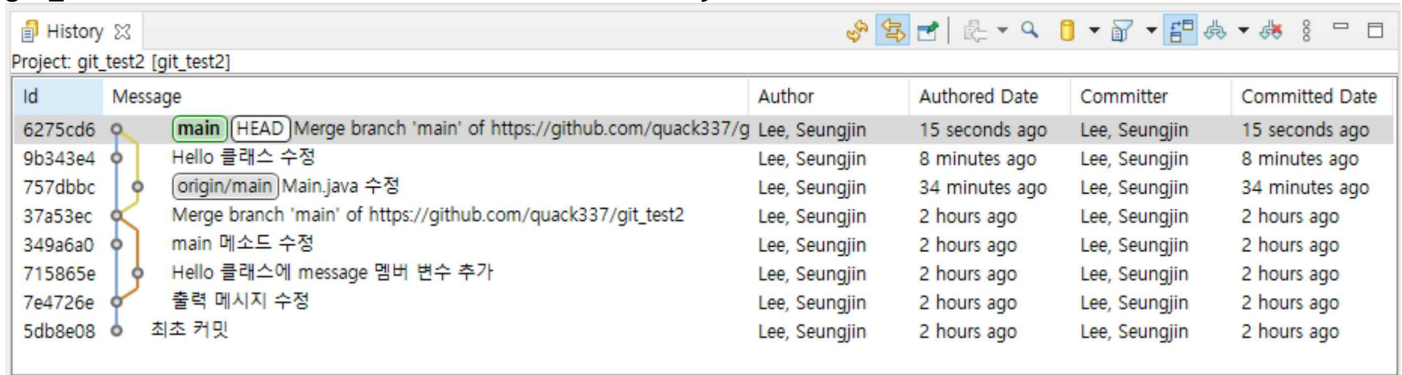
eclipse GUI local repository



7) history

history 창을 새로 열자

git_test2 프로젝트 우클릭 - Team - Show in History



Id	Message	Author	Authored Date	Committer	Committed Date
6275cd6	main (HEAD) Merge branch 'main' of https://github.com/quack337/g	Lee, Seungjin	15 seconds ago	Lee, Seungjin	15 seconds ago
9b343e4	Hello 클래스 수정	Lee, Seungjin	8 minutes ago	Lee, Seungjin	8 minutes ago
757dbbc	origin/main Main.java 수정	Lee, Seungjin	34 minutes ago	Lee, Seungjin	34 minutes ago
37a53ec	Merge branch 'main' of https://github.com/quack337/git_test2	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago
349a6a0	main 메소드 수정	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago
715865e	Hello 클래스에 message 멤버 변수 추가	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago
7e4726e	출력 메시지 수정	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago
5db8e08	최초 커밋	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago

"Hello 클래스 수정" => local repository 에만 등록된 commit

"Main.java 수정" => remote repository 에서 다운로드한 최신 commit

위 두 commit 이 수동 merge 되어,

"Merge branch 'main' of ..." commit 이 생성되어 local repository에 등록되었다.

"Hello 클래스 수정"

"Merge branch 'main' of ..."

이 두 commit 은 아직 push 되지 않았다.

8) push

git push

```
Terminal
MINGW64:/c/PJ/STS/java/git_test2

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git push
Enumerating objects: 21, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (11/11), 1.08 KiB | 1.08 MiB/s, done.
Total 11 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/quack337/git_test2.git
757dbbc..6275cd6 main -> main

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$
```

push 성공

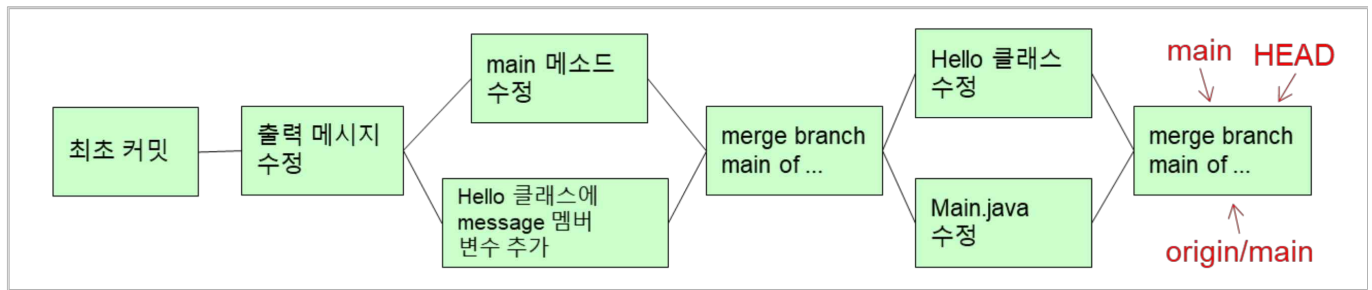
9) history

history 창을 새로 열자

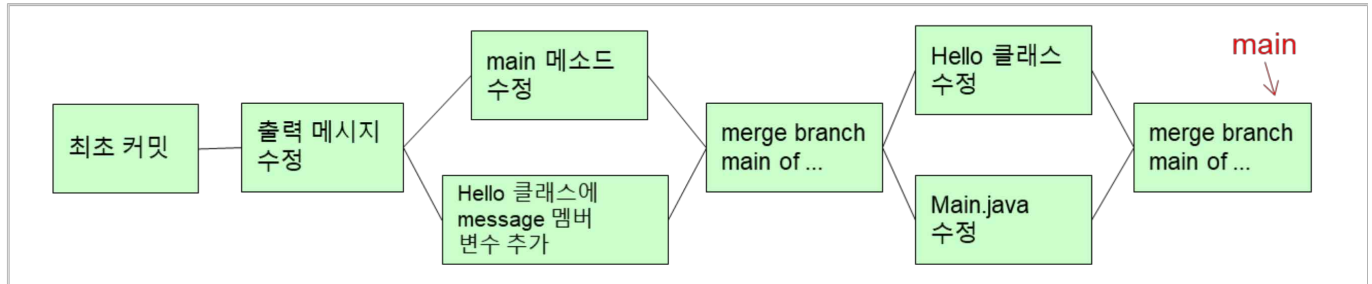
History						
Project: git_test2 [git_test2]						
Id	Message	Author	Authored Date	Committer	Committed Date	
6275cd6	main origin/main HEAD Merge branch 'main' of https://github.com	Lee, Seungjin	5 minutes ago	Lee, Seungjin	5 minutes ago	
9b343e4	Hello 클래스 수정	Lee, Seungjin	13 minutes ago	Lee, Seungjin	13 minutes ago	
757dbbc	Main.java 수정	Lee, Seungjin	39 minutes ago	Lee, Seungjin	39 minutes ago	
37a53ec	Merge branch 'main' of https://github.com/quack337/git_test2	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago	
349a6a0	main 메소드 수정	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago	
715865e	Hello 클래스에 message 멤버 변수 추가	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago	
7e4726e	출력 메시지 수정	Lee, Seungjin	2 hours ago	Lee, Seungjin	2 hours ago	
5db8e08	최초 커밋	Lee, Seungjin	3 hours ago	Lee, Seungjin	3 hours ago	

push 하기 전 history와 비교할 때 달라진 부분은?

eclipse GUI local repository



remote repository



4. 실습 #3 - VSCode

방금 eclipse 환경의 학생이 새 버전을 push 했으므로, vscode 환경의 학생은 수정 작업을 하기 전에 pull 부터 하는 것이 바람직하다.

그런데, merge conflict 상황을 다시 또 만들기 위해, pull 하지 않고 소스 코드를 수정하자.

1) Hello.java 수정

Hello.java

```
package git_test2;

public class Hello {

    String message = "안녕하세요";

    public String say() {
        return message;
    }

}
```

2) Main.java 수정

Main.java

```
package git_test2;

public class Main {

    public static void main(String[] args) {
        Hello hello = new Hello();
        System.out.println(hello.say());
    }

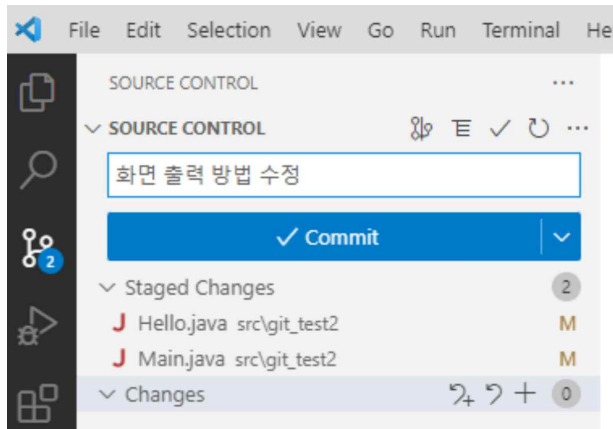
}
```

3) Commit

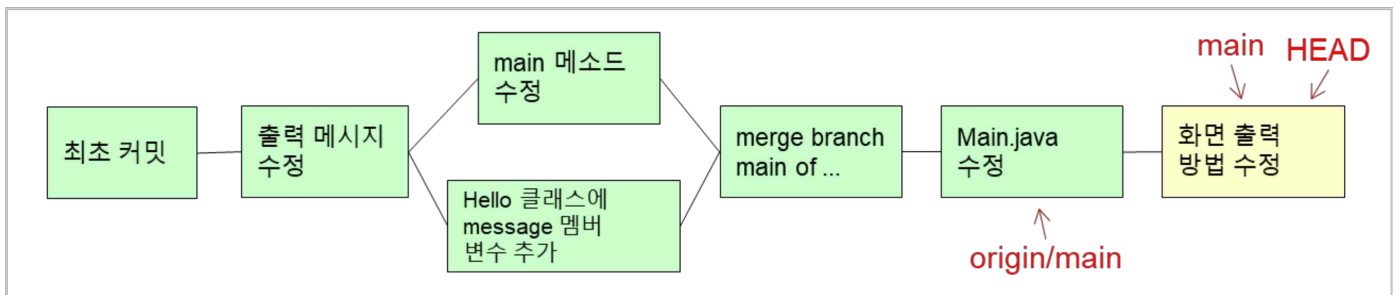
VSCode에서 Main.java와 Hello.java를 Staging Area에 선택하고 커밋하자.

커밋 메시지는

화면 출력 방법 수정



VSCode local repository



4) push 실패

git push

```
MINGW64:/c/PG/studentB/git_test2/src/git_test2
Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$ git push
To https://github.com/quack337/git_test2.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/quack337/git_test2.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$
```

push 하기 전에 pull 부터 해야 한다.

5) pull

git pull

```
MINGW64:/c/PG/studentB/git_test2/src/git_test2
Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$ git pull
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 11 (delta 2), reused 11 (delta 2), pack-reused 0
Unpacking objects: 100% (11/11), 1.06 KiB | 29.00 KiB/s, done.
From https://github.com/quack337/git_test2
  757dbbc..6275cd6  main    -> origin/main
Auto-merging src/git_test2/Main.java
CONFLICT (content): Merge conflict in src/git_test2/Main.java
Auto-merging src/git_test2/Hello.java
Automatic merge failed; fix conflicts and then commit the result.

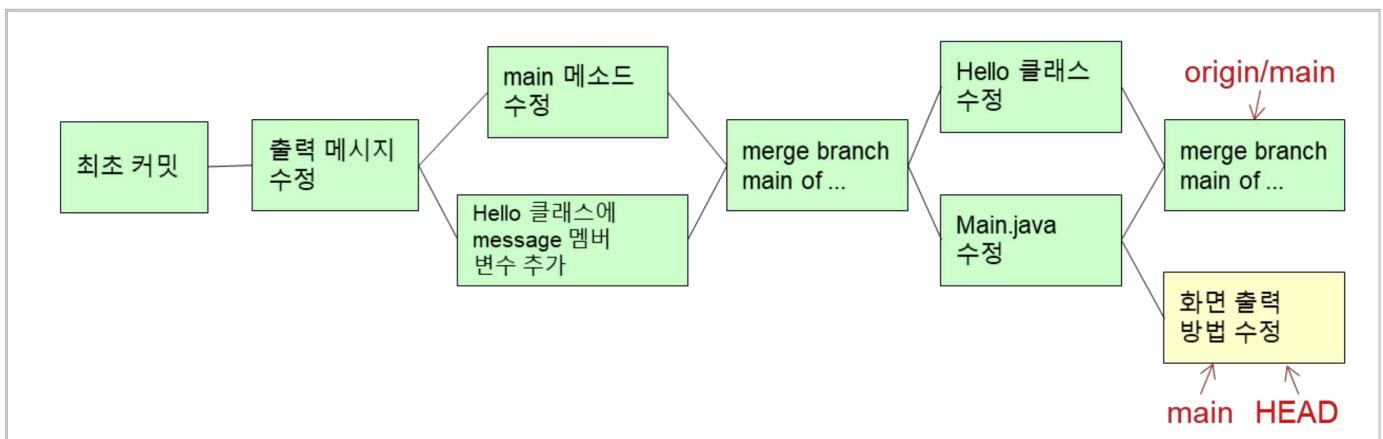
Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main|MERGING)
$
```

merge conflict 발생

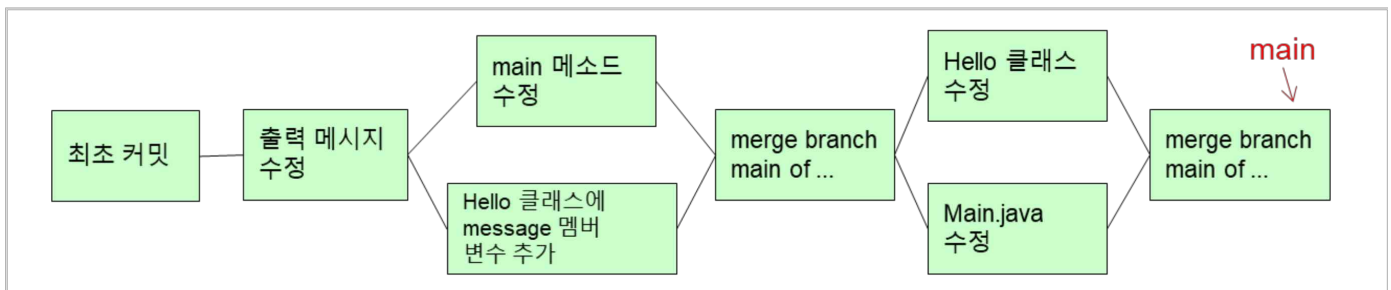
src/git_test2/Main.java 파일에서 자동 merge 실패.

src/git_test2/Hello.java 파일의 자동 merge는 성공

VSCode local repository



remote repository



Hello.java

```
package git_test2;

public class Hello {

    String message;

    public Hello() {
        this("안녕하세요");
    }

    public Hello(String message) {
        this.message = message;
    }

    public String say() {
        return message;
    }

}
```

충돌이 발생하지 않고 잘 병합되었다.

Main.java

```
package git_test2;

public class Main {

    public static void main(String[] args) {
<<<<<<< HEAD
        Hello hello = new Hello();
        System.out.println(hello.say());
=====
        Hello hello = new Hello("hello world");
        hello.say();
>>>>>>> 6275cd649ce48ad96567c9a26e8f7e6b193a2ed2
    }

}
```

merge conflict 발생

```
<<<<<<< HEAD
    local repository에만 commit된 소스코드
=====
    remote repository에서 다운로드한 최신 commit의 소스코드
>>>>>>> 6275cd649ce48ad96567c9a26e8f7e6b193a2ed2
```

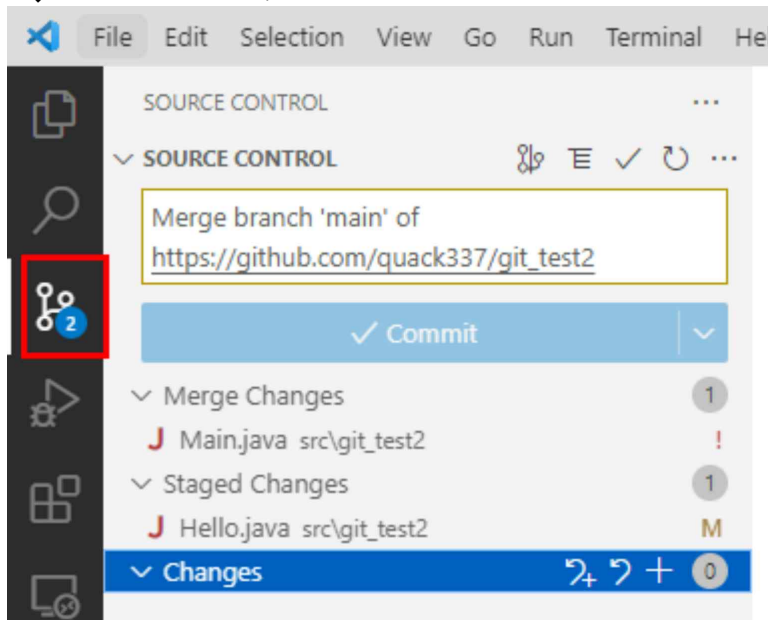
Main.java 수정

```
package git_test2;

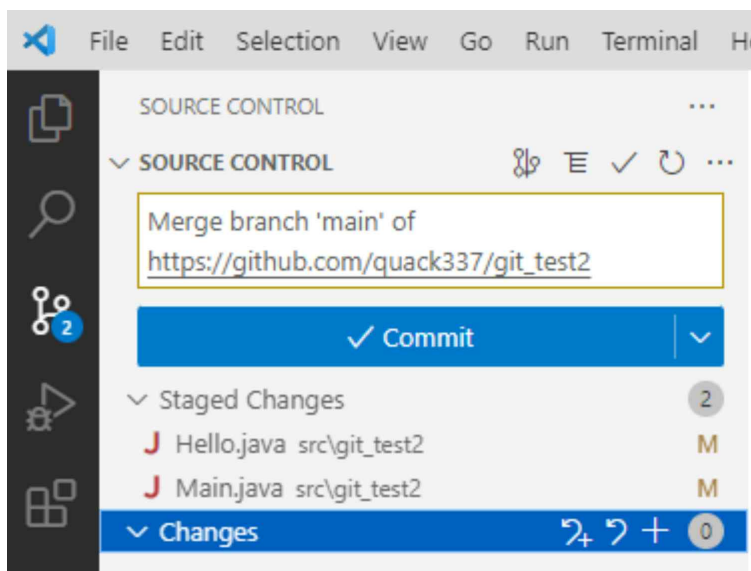
public class Main {

    public static void main(String[] args) {
        Hello hello = new Hello("hello world");
        System.out.println(hello.say());
    }
}
```

6) commit & push



자동 merge된 Hello.java는 이미 Staging Area에 등록되어 있고,
내가 수작업으로 병합한 Main.java는 Staging Area에 등록되어 있지 않다.
Main.java 옆의 + 버튼을 클릭해서 등록하자.



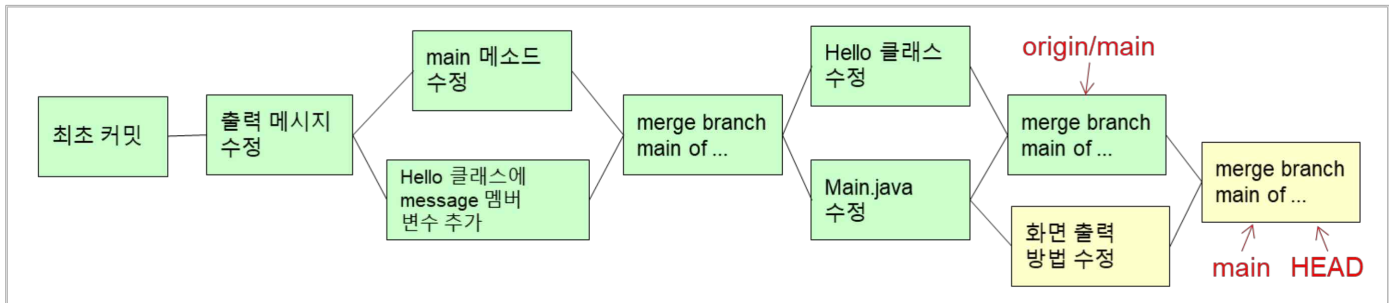
Commit 버튼 클릭

위 내용 중에서 # 문자로 시작되는 줄은, commit 메시지로 저장되지 않고 무시된다.

위 내용을 수정하지 않고, 편집기를 닫자.

편집기를 닫으면 commit이 완료된다.

VSCoDe local repository



push

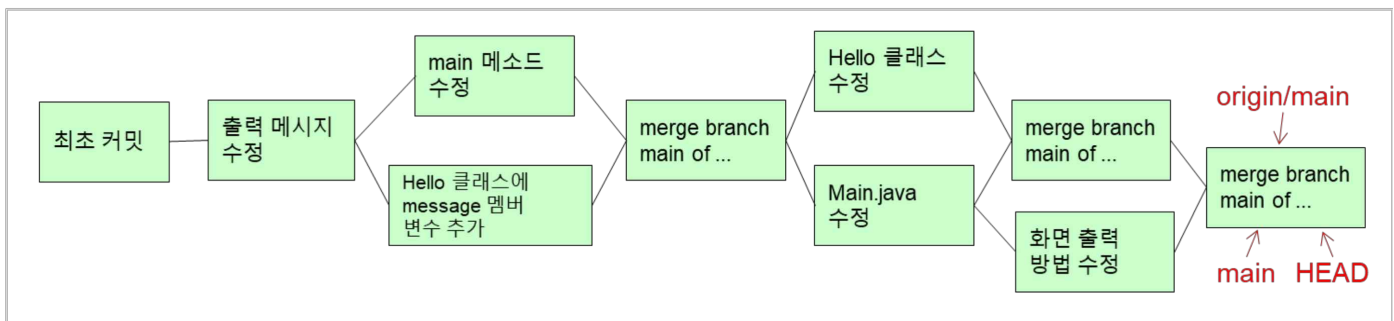
git push

```
Seungjin@LeeSeungJin2 MINGW64 /d/PJ/git/studentB/git_test2 (main)
$ git push
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (12/12), 1.04 KiB | 534.00 KiB/s, done.
Total 12 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/quack337/git_test2.git
6359110..b252c67 main -> main
```

Seungjin@LeeSeungJin2 MINGW64 /d/PJ/git/studentB/git_test2 (main)

push 성공

VSCoDe local repository



7) git log --graph

그래프 형태로 commit 목록 보기

```
git log --graph
```

```
MINGW64:/c/PG/studentB/git_test2/src/git_test2
Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$ git log --graph
* commit 21f69f32e1564a56e423fc9086440ef4dc303079 (HEAD -> main, origin/main, origin/HEAD)
  Merge: af0017c 6275cd6
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 15:07:31 2021 +0900

    Merge branch 'main' of https://github.com/quack337/git_test2

* commit 6275cd649ce48ad96567c9a26e8f7e6b193a2ed2
  Merge: 9b343e4 757dbbc
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 14:40:16 2021 +0900

    Merge branch 'main' of https://github.com/quack337/git_test2

    # Conflicts:
    #   src/git_test2/Main.java

* commit 9b343e43b28e8a4f35d860a9f54d4e30cf413e16
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 14:32:08 2021 +0900

    Hello 클래스 수정

* commit af0017c800a023e18a5c52527101c5492b1602c6
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 14:56:04 2021 +0900

    화면 출력 방법 수정

* commit 757dbbc31da27755a61648b874b85468314d2d6a
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 14:06:22 2021 +0900

    Main.java 수정

* commit 37a53ece664e22e1b0a5b7a814f8c1af8c6361cc
  Merge: 349a6a0 715865e
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 12:39:34 2021 +0900

    Merge branch 'main' of https://github.com/quack337/git_test2

* commit 715865edfff87936c5a3363a50c81206a8a7ad3a
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 12:26:51 2021 +0900

    Hello 클래스에 message 멤버 변수 추가

* commit 349a6a03ab52f0b132448cac521b3da3d33641e2
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 12:29:31 2021 +0900

    main 메소드 수정

* commit 7e4726e4d12ca246262b375640a8d75bfa87af96
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 12:15:32 2021 +0900

    출력 메시지 수정

* commit 5db8e0831ed42333f824760cb5ae475a80ab874d
  Author: Lee, Seungjin <quack337@gmail.com>
  Date: Tue Sep 7 12:14:29 2021 +0900

    최초 커밋

Seungjin@ryzen5 MINGW64 /c/PG/studentB/git_test2/src/git_test2 (main)
$
```


5. 실습 #4 - eclipse

1) pull

동료 팀원이 push한 내용을 pull 하자.

```
git pull
```

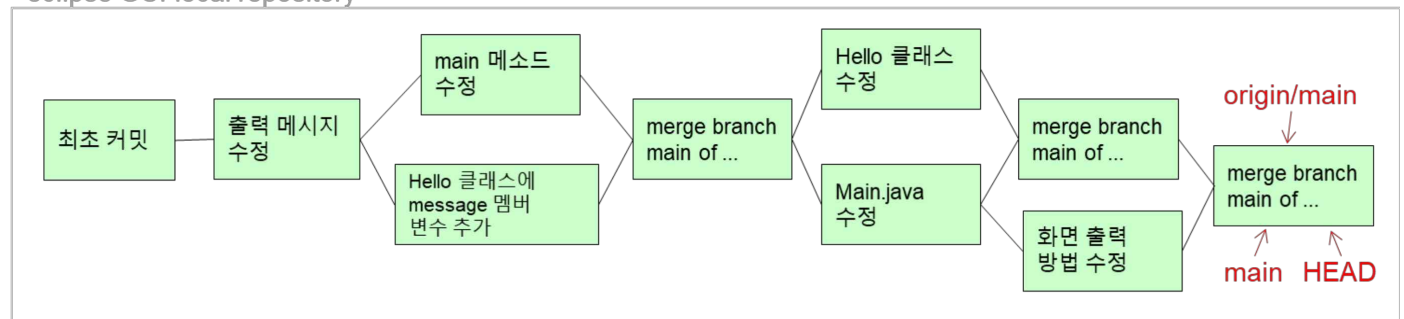
```
Terminal
MINGW64:/c/PJ/STS/java/git_test2

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$ git pull
remote: Enumerating objects: 22, done.
remote: Counting objects: 100% (22/22), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 12 (delta 3), reused 12 (delta 3), pack-reused 0
Unpacking objects: 100% (12/12), 1.09 KiB | 24.00 KiB/s, done.
From https://github.com/quack337/git_test2
  6275cd6..21f69f3  main      -> origin/main
Updating 6275cd6..21f69f3
Fast-forward
 src/git_test2/Hello.java | 4 ++--
 src/git_test2/Main.java  | 3 ++-
 2 files changed, 4 insertions(+), 3 deletions(-)

Seungjin@ryzen5 MINGW64 /c/PJ/STS/java/git_test2 (main)
$
```

성공

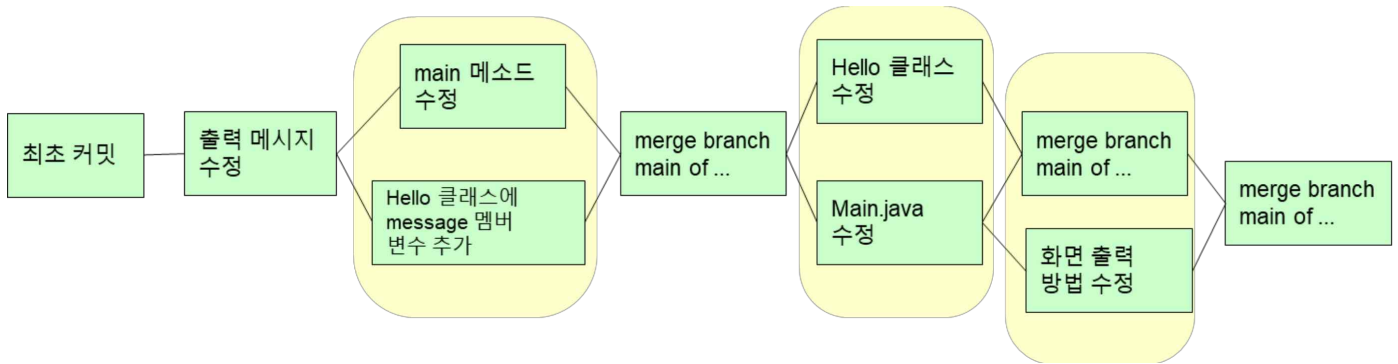
eclipse GUI local repository



2) history

History							
Project: git_test2 [git_test2]							
Id	Message	Author	Authored Date	Committer	Committed Date		
21f69f3	[main] [origin/main] [HEAD] Merge branch 'main' of https://github.com/quack337/git_test2	Lee, Seungjin	10 minutes ago	Lee, Seungjin	10 minutes ago		
af0017c	화면 출력 방법 수정	Lee, Seungjin	22 minutes ago	Lee, Seungjin	22 minutes ago		
6275cd6	Merge branch 'main' of https://github.com/quack337/git_test2	Lee, Seungjin	38 minutes ago	Lee, Seungjin	38 minutes ago		
9b343e4	Hello 클래스 수정	Lee, Seungjin	46 minutes ago	Lee, Seungjin	46 minutes ago		
757dbbc	Main.java 수정	Lee, Seungjin	72 minutes ago	Lee, Seungjin	72 minutes ago		
37a53ec	Merge branch 'main' of https://github.com/quack337/git_test2	Lee, Seungjin	3 hours ago	Lee, Seungjin	3 hours ago		
349a6a0	main 메소드 수정	Lee, Seungjin	3 hours ago	Lee, Seungjin	3 hours ago		
715865e	Hello 클래스에 message 멤버 변수 추가	Lee, Seungjin	3 hours ago	Lee, Seungjin	3 hours ago		
7e4726e	출력 메시지 수정	Lee, Seungjin	3 hours ago	Lee, Seungjin	3 hours ago		
5db8e08	최초 커밋	Lee, Seungjin	3 hours ago	Lee, Seungjin	3 hours ago		

6. 공동 작업



노란색으로 배경이 칠해진 한 쌍의 작업은 각각
eclipse GUI 개발 환경의 학생과
git CLI 개발 환경의 학생에 의해서
동시에 실행될 수 있다.

그리고 그 작업 결과는 자동 병합 되거나 수동 병합된다.

7. 요약

git commit 명령은 local repository 에만 새 commit을 등록한다.
local repository에 등록된 commit을 업로드 하려면, git push 명령을 실행해야 한다.

git push 하려면, 먼저 git pull 부터 해야 한다

git pull 명령 실행은 다음 3 개의 상황으로 전개된다.

- (1) 아직 push되지 않은 commit이, 내 local repository에 없다면,
최신 commit 들만 다운로드 되고 끝난다.
- (2) 아직 push되지 않은 commit이, 내 local repository에 있다면,
그 commit과 다운로드된 최신 commit이 자동으로 합쳐지고 (merge)
그 결과 새 commit이 만들어진다.
- (3) 아직 push되지 않은 commit이, 내 local repository에 있다면,
그 commit과 다운로드된 최신 commit이 자동으로 합쳐져야 하는데 (merge)
이 자동 merge 작업이 실패할 수도 있다. (merge conflict)

소스코드 파일에서 merge conflict 발생한 부분

```
<<<<<< HEAD
local repository 에만 commit 된 소스코드
=====
remote repository 에서 다운로드한 최신 commit 의 소스코드
>>>>>> 6275cd649ce48ad96567c9a26e8f7e6b193a2ed2
```

이 두 부분을 직접 수정해서 병합하고
commit 하고
push 해야 한다.